

Chapter 35 — 1D Dynamic Programming

0. What You Will Learn in This Chapter

1D Dynamic Programming is the most common starting point for DP interviews. A 1D DP problem usually has a state like:

dp[i]

or:

dp[amount]

or:

dp[index]

The answer depends on previous positions, previous amounts, or previous decisions.

By the end of this chapter, you should understand:

- What 1D DP means
- How to define dp[i]
- How to derive transitions from previous indices
- Fibonacci-style DP
- Choose/skip DP
- Min/max/count/boolean 1D DP
- Amount-based DP
- Prefix-based DP
- Subsequence-ending DP
- State-machine 1D DP
- Space optimization
- When 1D DP becomes 2D DP
- Common loop-order mistakes

Core problems covered in this chapter:

1. Climbing Stairs
2. Min Cost Climbing Stairs
3. House Robber
4. House Robber II
5. Maximum Subarray
6. Maximum Product Subarray
7. Decode Ways
8. Word Break
9. Coin Change
10. Perfect Squares
11. Longest Increasing Subsequence
12. Number of Longest Increasing Subsequence
13. Best Time to Buy and Sell Stock with Cooldown
14. Delete and Earn
15. Partition Array for Maximum Sum
16. Jump Game, DP view
17. Jump Game II, DP view

1. What Is 1D Dynamic Programming?

1D DP means the main state is represented using one dimension.

Examples:

dp[i] = answer up to index i
dp[i] = answer ending at index i
dp[i] = number of ways to reach i
dp[amount] = answer for amount
dp[sum] = whether sum is possible

Example:

Climbing stairs:

dp[i] = number of ways to reach step i

House Robber:

dp[i] = maximum money from houses 0..i

Coin Change:

dp[a] = minimum coins needed to make amount a

2. Why 1D DP Exists

Many problems process a linear sequence:

- stairs
- array
- string
- amount values
- days
- houses
- positions

At each position, the answer depends on previous positions.

Example:

To reach stair i, you can come from:

i - 1
i - 2

So one array dp[i] is enough.

3. Core Need / Why Use 1D DP?

Use 1D DP when:

- there is a linear order

- each state depends on earlier states
- choices repeat
- greedy is not always safe
- recursion has overlapping subproblems

Example:

House Robber has choices:

rob current house
skip current house

This cannot be solved by always picking the biggest house.
Need DP.

4. Main Types of 1D DP

Fibonacci-style	dp[i] from dp[i-1], dp[i-2]	Climbing Stairs
Choose/skip	dp[i] best up to i	House Robber
Ending-at-index	dp[i] best ending at i	Maximum Subarray
Amount-based	dp[a] answer for amount	Coin Change
Boolean prefix	dp[i] possible for prefix	Word Break
Count ways	dp[i] number of ways	Decode Ways
Subsequence	dp[i] ending at i	LIS
State machine	several states per i	Stock Cooldown

5. When to Use 1D DP

Use 1D DP when the problem asks:

5.1 Number of Ways

Examples:

- Climbing Stairs
- Decode Ways
- Tribonacci

5.2 Minimum Cost / Minimum Count

Examples:

- Min Cost Climbing Stairs
- Coin Change
- Perfect Squares

5.3 Maximum Value

Examples:

- House Robber
- Maximum Subarray
- Delete and Earn

5.4 Possibility

Examples:

- Word Break
- Jump Game, DP view

5.5 Longest Sequence

Examples:

- Longest Increasing Subsequence
 - Number of LIS
-

6. When NOT to Use 1D DP

6.1 Need Two Independent Indices

Use 2D DP.

Examples:

- Longest Common Subsequence
- Edit Distance
- Interleaving String

6.2 Need Range [l, r]

Use Interval DP.

Examples:

- Burst Balloons
- Matrix Chain Multiplication

6.3 Need Subset Mask

Use Bitmask DP.

Examples:

- Traveling Salesman
- Shortest Path Visiting All Nodes

6.4 Greedy Has a Proof

Use greedy.

Examples:

- Jump Game optimized
 - Best Time to Buy and Sell Stock II
-

7. Mental Models

7.1 DP as Memory

$dp[i]$ remembers the best answer for a smaller prefix or ending position.

7.2 Standing at Index i

Ask:

How could I have arrived here?
What previous states can lead here?

7.3 Choose or Skip

At each item:

take it
do not take it

7.4 Ending Here vs Up To Here

This distinction is crucial.

$dp[i]$ = best ending exactly at i

is different from:

$dp[i]$ = best among $0..i$

Maximum Subarray uses ending exactly at i .

House Robber uses best up to i .

8. Pattern Recognition Checklist

Ask:

19. Is input linear?
20. Does answer at index i depend on earlier indices?
21. Is there a repeated recurrence?
22. Are there choices at each position?
23. Is the problem asking min/max/count/possible?
24. Can I define $dp[i]$ clearly?
25. Is greedy unsafe?
26. Does recursion repeat states?
27. Can the state be represented with one variable?
28. Can space be optimized to a few variables?

9. C++ Tools Needed

9.1 Vector DP

```
vector<int> dp(n + 1, 0);
```

9.2 INF Initialization

```
const int INF = 1e9;  
vector<int> dp(amount + 1, INF);
```

9.3 Boolean DP

```
vector<bool> dp(n + 1, false);
```

9.4 Long Long for Counts

```
vector<long long> dp(n + 1, 0);
```

9.5 Space Variables

```
int prev2 = 0;  
int prev1 = 1;
```

10. Core Templates

Template 1 — Fibonacci-Style DP

```
int solve(int n) {
    if (n <= 1) {
        return n;
    }

    vector<int> dp(n + 1);

    dp[0] = 0;
    dp[1] = 1;

    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }

    return dp[n];
}
```

Template 2 — Space-Optimized Fibonacci-Style DP

```
int prev2 = base0;
int prev1 = base1;

for (int i = 2; i <= n; i++) {
    int curr = combine(prev1, prev2);

    prev2 = prev1;
    prev1 = curr;
}

return prev1;
```

Template 3 — Choose / Skip DP

```
vector<int> dp(n, 0);

dp[0] = nums[0];

for (int i = 1; i < n; i++) {
```

```

int skip = dp[i - 1];
int take = nums[i];

if (i >= 2) {
    take += dp[i - 2];
}

dp[i] = max(skip, take);
}

return dp[n - 1];

```

Template 4 — Ending-at-Index DP

```

dp[0] = nums[0];
answer = dp[0];

for (int i = 1; i < n; i++) {
    dp[i] = max(nums[i], dp[i - 1] + nums[i]);
    answer = max(answer, dp[i]);
}

```

Template 5 — Amount Minimum DP

```

vector<int> dp(amount + 1, INF);
dp[0] = 0;

for (int a = 1; a <= amount; a++) {
    for (int coin : coins) {
        if (a >= coin && dp[a - coin] != INF) {
            dp[a] = min(dp[a], dp[a - coin] + 1);
        }
    }
}

```

Template 6 — Boolean Prefix DP

```

vector<bool> dp(n + 1, false);
dp[0] = true;

for (int i = 1; i <= n; i++) {
    for (int j = 0; j < i; j++) {
        if (dp[j] && valid(j, i)) {

```

```

        dp[i] = true;
        break;
    }
}
}

```

Template 7 — LIS $O(n^2)$

```

vector<int> dp(n, 1);
int answer = 1;

for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        if (nums[j] < nums[i]) {
            dp[i] = max(dp[i], dp[j] + 1);
        }
    }

    answer = max(answer, dp[i]);
}

```

11. Major 1D DP Variations

Ways to reach	dp[i] ways to i	Climbing Stairs
Min cost	dp[i] min cost to i	Min Cost Stairs
Choose/skip	dp[i] best up to i	House Robber
Ending here	dp[i] best ending at i	Maximum Subarray
Amount min	dp[a] min coins	Coin Change
Prefix valid	dp[i] prefix possible	Word Break
Decoding count	dp[i] decode first i chars	Decode Ways
Subsequence ending	dp[i] best sequence ending i	LIS
State machine	hold/sold/rest	Stock Cooldown

12. Full Problem Walkthrough 1 — LeetCode 70: Climbing Stairs

12.1 Problem Summary

You can climb 1 or 2 steps.

Return number of distinct ways to reach step n.

12.2 How to Identify 1D DP

Number of ways to reach step i depends on previous steps.

12.3 State

$dp[i]$ = number of ways to reach step i

12.4 Transition

$dp[i] = dp[i - 1] + dp[i - 2]$

12.5 Base Cases

$dp[1] = 1$

$dp[2] = 2$

12.6 C++ Code

```
class Solution {
public:
    int climbStairs(int n) {
        if (n <= 2) {
            return n;
        }

        int prev2 = 1;
        int prev1 = 2;

        for (int i = 3; i <= n; i++) {
            int curr = prev1 + prev2;

            prev2 = prev1;
            prev1 = curr;
        }

        return prev1;
    }
};
```

12.7 Complexity

Time:

$O(n)$

Space:

$O(1)$

13. Full Problem Walkthrough 2 — LeetCode 746: Min Cost Climbing Stairs

13.1 Problem Summary

You can climb 1 or 2 steps.

Each step has a cost.

You can start at index 0 or 1.

Return minimum cost to reach the top.

13.2 State

$dp[i]$ = minimum cost to reach step i

Top is step n .

13.3 Transition

$$dp[i] = \min(dp[i - 1] + cost[i - 1], dp[i - 2] + cost[i - 2])$$

13.4 Base Cases

$dp[0] = 0$

$dp[1] = 0$

13.5 C++ Code

```
class Solution {
public:
    int minCostClimbingStairs(vector<int>& cost) {
        int n = cost.size();

        int prev2 = 0;
        int prev1 = 0;

        for (int i = 2; i <= n; i++) {
            int curr = min(prev1 + cost[i - 1],
                           prev2 + cost[i - 2]);

            prev2 = prev1;
            prev1 = curr;
        }
    }
};
```

```
    }  
  
    return prev1;  
}  
};
```

13.6 Complexity

Time:

$O(n)$

Space:

$O(1)$

14. Full Problem Walkthrough 3 — LeetCode 198: House Robber

14.1 Problem Summary

You cannot rob adjacent houses.
Return maximum money.

14.2 State

$dp[i]$ = maximum money from houses $0..i$

14.3 Transition

At house i :

```
skip = dp[i - 1]  
take = dp[i - 2] + nums[i]
```

So:

$dp[i] = \max(\text{skip}, \text{take})$

14.4 C++ Code

```
class Solution {  
public:  
    int rob(vector<int>& nums) {  
        int prev2 = 0;  
        int prev1 = 0;  
  
        for (int money : nums) {  
            int curr = max(prev1, prev2 + money);  
  
            prev2 = prev1;  
            prev1 = curr;  
        }  
    }  
};
```

```
    }  
  
    return prev1;  
}  
};
```

14.5 Complexity

Time:

$O(n)$

Space:

$O(1)$

15. Full Problem Walkthrough 4 — LeetCode 213: House Robber II

15.1 Problem Summary

Houses are in a circle.

Cannot rob adjacent houses.

First and last houses are adjacent.

15.2 How to Identify 1D DP

It is House Robber with circular constraint.

Cannot take both first and last.

Solve two linear cases:

29. Rob houses 0..n-2

30. Rob houses 1..n-1

15.3 C++ Code

```
class Solution {  
public:  
    int robLinear(vector<int>& nums, int left, int right) {  
        int prev2 = 0;  
        int prev1 = 0;  
  
        for (int i = left; i <= right; i++) {  
            int curr = max(prev1, prev2 + nums[i]);  
  
            prev2 = prev1;  
            prev1 = curr;  
        }  
  
        return prev1;  
}
```

```
int rob(vector<int>& nums) {  
    int n = nums.size();  
  
    if (n == 1) {  
        return nums[0];  
    }  
  
    return max(robLinear(nums, 0, n - 2),  
              robLinear(nums, 1, n - 1));  
}  
};
```

15.4 Complexity

Time:

$O(n)$

Space:

$O(1)$

16. Full Problem Walkthrough 5 — LeetCode 53: Maximum Subarray

16.1 Problem Summary

Find maximum sum of a contiguous subarray.

16.2 State

$dp[i]$ = maximum subarray sum ending exactly at i

16.3 Transition

Either start new subarray at i :

$nums[i]$

or extend previous subarray:

$dp[i - 1] + nums[i]$

So:

$dp[i] = \max(nums[i], dp[i - 1] + nums[i])$

16.4 C++ Code

```
class Solution {  
public:  
    int maxSubArray(vector<int>& nums) {  
        int current = nums[0];
```

```
int best = nums[0];

for (int i = 1; i < nums.size(); i++) {
    current = max(nums[i], current + nums[i]);
    best = max(best, current);
}

return best;
};
```

16.5 Complexity

Time:

$O(n)$

Space:

$O(1)$

16.6 Common Mistake

Initializing best = 0 fails for all-negative arrays.

17. Full Problem Walkthrough 6 — LeetCode 152: Maximum Product Subarray

17.1 Problem Summary

Find contiguous subarray with maximum product.

17.2 Why This Is Trickier Than Maximum Sum

A negative number can turn the smallest negative product into the largest positive product.

So track both:

currentMax
currentMin

17.3 State

currentMax = maximum product ending here
currentMin = minimum product ending here

17.4 C++ Code

```
class Solution {
public:
    int maxProduct(vector<int>& nums) {
```

```

int currentMax = nums[0];
int currentMin = nums[0];
int best = nums[0];

for (int i = 1; i < nums.size(); i++) {
    int x = nums[i];

    if (x < 0) {
        swap(currentMax, currentMin);
    }

    currentMax = max(x, currentMax * x);
    currentMin = min(x, currentMin * x);

    best = max(best, currentMax);
}

return best;
};

```

17.5 Complexity

Time:

$O(n)$

Space:

$O(1)$

18. Full Problem Walkthrough 7 — LeetCode 91: Decode Ways

18.1 Problem Summary

Decode digit string using:

1 -> A
...
26 -> Z

Return number of decodings.

18.2 State

$dp[i]$ = number of ways to decode first i characters

18.3 Transition

Single digit:

```
if s[i - 1] != '0':  
    dp[i] += dp[i - 1]
```

Two digits:

```
if 10 <= value <= 26:  
    dp[i] += dp[i - 2]
```

18.4 C++ Code

```
class Solution {  
public:  
    int numDecodings(string s) {  
        int n = s.size();  
  
        vector<int> dp(n + 1, 0);  
        dp[0] = 1;  
  
        for (int i = 1; i <= n; i++) {  
            if (s[i - 1] != '0') {  
                dp[i] += dp[i - 1];  
            }  
  
            if (i >= 2) {  
                int value = (s[i - 2] - '0') * 10 + (s[i - 1] - '0');  
  
                if (value >= 10 && value <= 26) {  
                    dp[i] += dp[i - 2];  
                }  
            }  
        }  
  
        return dp[n];  
    }  
};
```

18.5 Complexity

Time:

$O(n)$

Space:

$O(n)$

Can be optimized to $O(1)$.

19. Full Problem Walkthrough 8 —

LeetCode 139: Word Break

19.1 Problem Summary

Given string and dictionary, return true if string can be segmented into dictionary words.

19.2 State

$dp[i]$ = true if $s[0..i-1]$ can be segmented

19.3 Transition

For any $j < i$:

if $dp[j]$ and $s[j..i-1]$ in dictionary:
 $dp[i]$ = true

19.4 C++ Code

```
class Solution {
public:
    bool wordBreak(string s, vector<string>& wordDict) {
        unordered_set<string> dict(wordDict.begin(), wordDict.end());

        int n = s.size();

        vector<bool> dp(n + 1, false);
        dp[0] = true;

        for (int i = 1; i <= n; i++) {
            for (int j = 0; j < i; j++) {
                if (dp[j] && dict.count(s.substr(j, i - j))) {
                    dp[i] = true;
                    break;
                }
            }
        }

        return dp[n];
    }
};
```

19.5 Complexity

Time:

$O(n^3)$

because substring creation costs time.

With optimization, can improve.

20. Full Problem Walkthrough 9 — LeetCode 322: Coin Change

20.1 Problem Summary

Return minimum number of coins needed to make amount.

If impossible, return -1.

20.2 State

$dp[a]$ = minimum coins needed to make amount a

20.3 Transition

For each coin:

$dp[a] = \min(dp[a], dp[a - \text{coin}] + 1)$

20.4 C++ Code

```
class Solution {
public:
    int coinChange(vector<int>& coins, int amount) {
        const int INF = 1e9;

        vector<int> dp(amount + 1, INF);

        dp[0] = 0;

        for (int a = 1; a <= amount; a++) {
            for (int coin : coins) {
                if (a >= coin && dp[a - coin] != INF) {
                    dp[a] = min(dp[a], dp[a - coin] + 1);
                }
            }
        }

        return dp[amount] == INF ? -1 : dp[amount];
    }
};
```

20.5 Complexity

Time:

$O(\text{amount} * \text{coins})$

Space:

O(amount)

21. Full Problem Walkthrough 10 — LeetCode 279: Perfect Squares

21.1 Problem Summary

Return minimum number of perfect squares that sum to n.

Example:

n = 12

Output:

3

because:

4 + 4 + 4

21.2 DP Similarity

This is like Coin Change.

Coins are perfect squares:

1, 4, 9, 16, ...

21.3 C++ Code

```
class Solution {
public:
    int numSquares(int n) {
        const int INF = 1e9;

        vector<int> dp(n + 1, INF);
        dp[0] = 0;

        for (int amount = 1; amount <= n; amount++) {
            for (int square = 1; square * square <= amount; square++) {
                dp[amount] = min(dp[amount],
                                dp[amount - square * square] + 1);
            }
        }

        return dp[n];
    }
};
```

21.4 Complexity

Time:

$O(n * \sqrt{n})$

Space:

$O(n)$

22. Full Problem Walkthrough 11 — LeetCode 300: Longest Increasing Subsequence

22.1 Problem Summary

Return length of longest strictly increasing subsequence.

22.2 State

$dp[i]$ = length of LIS ending exactly at i

22.3 Transition

For all $j < i$:

```
if nums[j] < nums[i]:  
    dp[i] = max(dp[i], dp[j] + 1)
```

22.4 C++ Code $O(n^2)$

```
class Solution {  
public:  
    int lengthOfLIS(vector<int>& nums) {  
        int n = nums.size();  
  
        vector<int> dp(n, 1);  
  
        int answer = 1;  
  
        for (int i = 0; i < n; i++) {  
            for (int j = 0; j < i; j++) {  
                if (nums[j] < nums[i]) {  
                    dp[i] = max(dp[i], dp[j] + 1);  
                }  
            }  
  
            answer = max(answer, dp[i]);  
        }  
  
        return answer;  
    }  
};
```

```
}  
};
```

22.5 Complexity

Time:

$O(n^2)$

Space:

$O(n)$

23. Full Problem Walkthrough 12 — LeetCode 673: Number of Longest Increasing Subsequence

23.1 Problem Summary

Return number of longest increasing subsequences.

23.2 State

Need two arrays:

length[i] = length of LIS ending at i
count[i] = number of LIS ending at i

23.3 Transition

If $\text{nums}[j] < \text{nums}[i]$:
If better length found:

length[i] = length[j] + 1
count[i] = count[j]

If same best length found:

count[i] += count[j]

23.4 C++ Code

```
class Solution {  
public:  
    int findNumberOfLIS(vector<int>& nums) {  
        int n = nums.size();  
  
        vector<int> length(n, 1);  
        vector<int> count(n, 1);  
  
        int bestLength = 1;  
  
        for (int i = 0; i < n; i++) {
```

```

        for (int j = 0; j < i; j++) {
            if (nums[j] < nums[i]) {
                if (length[j] + 1 > length[i]) {
                    length[i] = length[j] + 1;
                    count[i] = count[j];
                } else if (length[j] + 1 == length[i]) {
                    count[i] += count[j];
                }
            }
        }

        bestLength = max(bestLength, length[i]);
    }

    int answer = 0;

    for (int i = 0; i < n; i++) {
        if (length[i] == bestLength) {
            answer += count[i];
        }
    }

    return answer;
}
};

```

23.5 Complexity

Time:

$O(n^2)$

Space:

$O(n)$

24. Full Problem Walkthrough 13 — LeetCode 309: Best Time to Buy and Sell Stock With Cooldown

24.1 Problem Summary

You may buy and sell stock.

After selling, you must wait one day before buying again.

Return max profit.

24.2 State

At each day, use three states:

hold = max profit while holding stock
sold = max profit just sold today
rest = max profit not holding and not just sold today

24.3 Transitions

hold = max(prevHold, prevRest - price)
sold = prevHold + price
rest = max(prevRest, prevSold)

24.4 C++ Code

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int hold = -prices[0];
        int sold = 0;
        int rest = 0;

        for (int i = 1; i < prices.size(); i++) {
            int prevHold = hold;
            int prevSold = sold;
            int prevRest = rest;

            hold = max(prevHold, prevRest - prices[i]);
            sold = prevHold + prices[i];
            rest = max(prevRest, prevSold);
        }

        return max(sold, rest);
    }
};
```

24.5 Complexity

Time:

O(n)

Space:

O(1)

25. Full Problem Walkthrough 14 —

LeetCode 740: Delete and Earn

25.1 Problem Summary

If you take number x , you earn $x * \text{frequency}[x]$.

But then you cannot take $x - 1$ or $x + 1$.

Return max points.

25.2 How to Identify 1D DP

This becomes House Robber over number values.

For each value x :

```
points[x] = x * count[x]
```

Then cannot take adjacent values.

25.3 C++ Code

```
class Solution {
public:
    int deleteAndEarn(vector<int>& nums) {
        int maxVal = 0;

        for (int x : nums) {
            maxVal = max(maxVal, x);
        }

        vector<int> points(maxVal + 1, 0);

        for (int x : nums) {
            points[x] += x;
        }

        int prev2 = 0;
        int prev1 = 0;

        for (int x = 0; x <= maxVal; x++) {
            int curr = max(prev1, prev2 + points[x]);

            prev2 = prev1;
            prev1 = curr;
        }

        return prev1;
    }
};
```

```
}  
};
```

25.4 Complexity

Time:

$O(n + \text{maxValue})$

Space:

$O(\text{maxValue})$

26. Full Problem Walkthrough 15 — LeetCode 1043: Partition Array for Maximum Sum

26.1 Problem Summary

Partition array into subarrays of length at most k .

Replace every element in a partition with the maximum of that partition.

Return maximum possible sum.

26.2 How to Identify DP

At index i , choose last partition length from 1 to k .

26.3 State

$dp[i]$ = maximum sum for first i elements

26.4 Transition

Last partition is from j to $i - 1$.

$dp[i] = \max(dp[j] + \max(arr[j..i-1]) * (i - j))$

26.5 C++ Code

```
class Solution {  
public:  
    int maxSumAfterPartitioning(vector<int>& arr, int k) {  
        int n = arr.size();  
  
        vector<int> dp(n + 1, 0);  
  
        for (int i = 1; i <= n; i++) {  
            int maxVal = 0;  
  
            for (int len = 1; len <= k && i - len >= 0; len++) {  
                maxVal = max(maxVal, arr[i - len]);  
            }  
            dp[i] = max(dp[i], dp[i - len] + maxVal * len);  
        }  
        return dp[n];  
    }  
};
```

```

        dp[i] = max(dp[i],
                    dp[i - len] + maxVal * len);
    }
}

return dp[n];
}
};

```

26.6 Complexity

Time:

$O(n * k)$

Space:

$O(n)$

27. Full Problem Walkthrough 16 — Jump Game, DP View

27.1 Problem Summary

Return true if last index is reachable.

27.2 State

$dp[i]$ = true if index i is reachable

27.3 Transition

For every reachable j , if:

$j + \text{nums}[j] \geq i$

then i is reachable.

27.4 C++ Code

```

class Solution {
public:
    bool canJump(vector<int>& nums) {
        int n = nums.size();

        vector<bool> dp(n, false);
        dp[0] = true;

        for (int i = 1; i < n; i++) {
            for (int j = 0; j < i; j++) {
                if (dp[j] && j + nums[j] >= i) {
                    dp[i] = true;
                }
            }
        }
    }
};

```

```

        break;
    }
}

return dp[n - 1];
}
};

```

27.5 Complexity

Time:

$O(n^2)$

Space:

$O(n)$

27.6 Note

Greedy $O(n)$ is better, but DP view teaches reachability state.

28. Full Problem Walkthrough 17 — Jump Game II, DP View

28.1 Problem Summary

Return minimum jumps to reach last index.

28.2 State

$dp[i]$ = minimum jumps needed to reach index i

28.3 Transition

For every previous index j that can reach i :

$dp[i] = \min(dp[i], dp[j] + 1)$

28.4 C++ Code

```

class Solution {
public:
    int jump(vector<int>& nums) {
        int n = nums.size();

        const int INF = 1e9;
        vector<int> dp(n, INF);

        dp[0] = 0;

        for (int i = 1; i < n; i++) {

```

```

        for (int j = 0; j < i; j++) {
            if (j + nums[j] >= i) {
                dp[i] = min(dp[i], dp[j] + 1);
            }
        }
    }

    return dp[n - 1];
}
};

```

28.5 Complexity

Time:

$O(n^2)$

Space:

$O(n)$

28.6 Note

Greedy $O(n)$ is optimal, but DP view is useful for understanding.

29. Full Problem Walkthrough 18 — Frog Jump Preview

29.1 Problem Summary

A frog crosses stones.

If last jump was k , next jump must be:

$k - 1$, k , or $k + 1$

Return true if frog can reach last stone.

29.2 Why This Is More Than Simple 1D DP

State needs:

stone position + last jump size

So it becomes:

$dp[index][jump]$

or hash-set DP.

This is a preview of 2D/state DP.

29.3 Lesson

If $dp[i]$ alone is not enough because the future depends on an extra variable, add another state dimension.

30. Common 1D DP Edge Cases

30.1 Empty Array

Return depends on problem.

30.2 One Element

Important for:

- House Robber II
- Maximum Subarray
- Maximum Product Subarray

30.3 All Negative Values

Initialize correctly.

30.4 Target 0

For amount DP:

$dp[0] = 0$ or true or 1

depending on meaning.

30.5 String Starts With 0

Decode Ways:

"0" -> 0

30.6 Impossible State

Use INF or false.

31. Common 1D DP Mistakes

Mistake 1 — Confusing Ending Here vs Best So Far

Maximum Subarray:

current = best ending here

best = best anywhere

Mistake 2 — Wrong Base Case

For Min Cost Stairs:

$dp[0] = 0$

$dp[1] = 0$

not cost values.

Mistake 3 — Wrong Loop Direction

For 0/1 knapsack-like problems, loop backward.

For unbounded problems, loop forward.

Mistake 4 — Using Greedy When DP Is Needed

Coin Change is not always greedy.

Mistake 5 — Not Checking Impossible State

Do not do:

$dp[a] = \min(dp[a], dp[a - \text{coin}] + 1);$

if $dp[a - \text{coin}]$ is INF and might overflow.

32. Interview Explanation Templates

32.1 Fibonacci-Style DP

I define $dp[i]$ as the answer for position i . Since position i can be reached from previous positions, I compute it from earlier states.

32.2 House Robber

At each house, I either skip it and keep the previous best, or rob it and add its money to the best before the adjacent house.

32.3 Maximum Subarray

I track the best subarray ending at the current index. Either I start fresh at current element or extend the previous subarray.

32.4 Coin Change

For every amount, I try every coin as the last coin used. If amount minus that coin is possible, I update the minimum coins for the current amount.

32.5 Word Break

I define $dp[i]$ as whether the prefix ending at i can be segmented. I try every split point j and check if the left prefix is valid and the right substring is a dictionary word.

33. Complexity Patterns

Climbing Stairs	$O(n)$	$O(1)$
Min Cost Stairs	$O(n)$	$O(1)$
House Robber	$O(n)$	$O(1)$
House Robber II	$O(n)$	$O(1)$
Maximum Subarray	$O(n)$	$O(1)$
Maximum Product Subarray	$O(n)$	$O(1)$
Decode Ways	$O(n)$	$O(n), O(1)$ possible
Word Break	$O(n^3)$ simple	$O(n)$
Coin Change	$O(\text{amount} * \text{coins})$	$O(\text{amount})$
Perfect Squares	$O(n \sqrt{n})$	$O(n)$
LIS $O(n^2)$	$O(n^2)$	$O(n)$
Number of LIS	$O(n^2)$	$O(n)$

Problem Type	Time	Space
Stock Cooldown	$O(n)$	$O(1)$
Delete and Earn	$O(n + \text{maxVal})$	$O(\text{maxVal})$
Partition Array Max Sum	$O(nk)$	$O(n)$

34. Time-Boxed Practice Guidance

Easy 1D DP

Target:

15–25 minutes

Problems:

- Climbing Stairs
- Min Cost Climbing Stairs
- Fibonacci
- Tribonacci

Medium 1D DP

Target:

25–45 minutes

Problems:

- House Robber
- Maximum Subarray
- Decode Ways
- Coin Change
- Word Break

Harder 1D DP

Target:

45–70 minutes

Problems:

- Maximum Product Subarray
- Number of LIS
- Stock Cooldown
- Partition Array for Maximum Sum
- Delete and Earn

35. Expanded Practice Set

35.1 Must-Do Easy

31. Fibonacci Number
32. N-th Tribonacci Number
33. Climbing Stairs
34. Min Cost Climbing Stairs
35. Pascal's Triangle II

35.2 Must-Do Medium

- 36. House Robber
- 37. House Robber II
- 38. Maximum Subarray
- 39. Maximum Product Subarray
- 40. Decode Ways
- 41. Word Break
- 42. Coin Change
- 43. Perfect Squares
- 44. Longest Increasing Subsequence
- 45. Delete and Earn
- 46. Partition Array for Maximum Sum

35.3 Strong Interview Practice

- 47. Number of Longest Increasing Subsequence
- 48. Best Time to Buy and Sell Stock With Cooldown
- 49. Best Time to Buy and Sell Stock With Transaction Fee
- 50. Wiggle Subsequence
- 51. Arithmetic Slices
- 52. Integer Break
- 53. Count Ways to Build Good Strings
- 54. Minimum Cost For Tickets
- 55. Solving Questions With Brainpower

35.4 Hard / Stretch

- 56. Frog Jump
 - 57. Race Car
 - 58. Count All Possible Routes
 - 59. Largest Sum of Averages
 - 60. Student Attendance Record II
 - 61. Paint Fence
 - 62. Decode Ways II
-

36. Suggested Study Order

Step 1 — Fibonacci-Style

Start with:

- 63. Fibonacci
- 64. Tribonacci
- 65. Climbing Stairs
- 66. Min Cost Climbing Stairs

Goal:

Learn previous-state transitions.

Step 2 — Choose / Skip

Solve:

- 67. House Robber
- 68. House Robber II
- 69. Delete and Earn

Goal:

Learn take/skip recurrence.

Step 3 — Ending-at-Index

Solve:

- 70. Maximum Subarray
- 71. Maximum Product Subarray
- 72. Wiggle Subsequence

Goal:

Understand state ending exactly at current index.

Step 4 — Amount DP

Solve:

- 73. Coin Change
- 74. Perfect Squares
- 75. Minimum Cost For Tickets

Goal:

Learn min-cost amount/state DP.

Step 5 — Prefix/String 1D DP

Solve:

- 76. Decode Ways
- 77. Word Break

Goal:

Learn prefix validity/count states.

Step 6 — Subsequence DP

Solve:

- 78. Longest Increasing Subsequence
- 79. Number of LIS

Goal:

Learn $O(n^2)$ transitions across previous indices.

Step 7 — State Machine DP

Solve:

- 80. Stock With Cooldown
- 81. Stock With Transaction Fee

Goal:

Learn multiple states per day/index.

37. Pattern Decision Table

ways to reach step	dp[i] ways	Climbing Stairs
min cost to top	dp[i] min cost	Min Cost Stairs
cannot take adjacent	dp[i] choose/skip	House Robber
circular adjacency	two linear DP runs	House Robber II
max subarray ending here	current ending state	Maximum Subarray
product with negatives	max/min ending state	Maximum Product

Problem Clue	State	Example
decode prefix	dp[i] ways for prefix	Decode Ways
segment string	dp[i] prefix possible	Word Break
min coins for amount	dp[a] min count	Coin Change
perfect square sum	dp[n] min count	Perfect Squares
increasing subsequence	dp[i] ending at i	LIS
count LIS	length + count arrays	Number of LIS
cooldown or holding	state machine	Stock Cooldown
choose values not adjacent	value-index robber	Delete and Earn

38. Mini Problem 1 — Wiggle Subsequence

```

class Solution {
public:
    int wiggleMaxLength(vector<int>& nums) {
        int up = 1;
        int down = 1;

        for (int i = 1; i < nums.size(); i++) {
            if (nums[i] > nums[i - 1]) {
                up = down + 1;
            } else if (nums[i] < nums[i - 1]) {
                down = up + 1;
            }
        }

        return max(up, down);
    }
};

```

State:

up = best wiggle ending with upward difference
down = best wiggle ending with downward difference

39. Mini Problem 2 — Minimum Cost For Tickets

```
class Solution {
public:
    int mincostTickets(vector<int>& days, vector<int>& costs) {
        unordered_set<int> travel(days.begin(), days.end());

        int lastDay = days.back();
        vector<int> dp(lastDay + 1, 0);

        for (int day = 1; day <= lastDay; day++) {
            if (!travel.count(day)) {
                dp[day] = dp[day - 1];
            } else {
                int one = dp[max(0, day - 1)] + costs[0];
                int seven = dp[max(0, day - 7)] + costs[1];
                int thirty = dp[max(0, day - 30)] + costs[2];

                dp[day] = min({one, seven, thirty});
            }
        }

        return dp[lastDay];
    }
};
```

40. Mini Problem 3 — Solving Questions With Brainpower

```
class Solution {
public:
    long long mostPoints(vector<vector<int>>& questions) {
        int n = questions.size();

        vector<long long> dp(n + 1, 0);

        for (int i = n - 1; i >= 0; i--) {
            long long points = questions[i][0];
```

```

        int brainpower = questions[i][1];

        int next = min(n, i + brainpower + 1);

        long long solve = points + dp[next];
        long long skip = dp[i + 1];

        dp[i] = max(solve, skip);
    }

    return dp[0];
}
};

```

State:

dp[i] = max points from questions i..n-1

41. Mini Problem 4 — Arithmetic Slices

```

class Solution {
public:
    int numberOfArithmeticSlices(vector<int>& nums) {
        int current = 0;
        int total = 0;

        for (int i = 2; i < nums.size(); i++) {
            if (nums[i] - nums[i - 1] == nums[i - 1] - nums[i - 2]) {
                current++;
                total += current;
            } else {
                current = 0;
            }
        }

        return total;
    }
};

```

State:

current = number of arithmetic subarrays ending at i

42. Common Edge Case Checklist

Before submitting 1D DP, test:

empty input
one element
two elements
all zeros
all negatives
target = 0
amount impossible
string begins with 0
string has 10 or 20
string has 06
circular array length 1
large amount
large max value

43. Debugging 1D DP

If your 1D DP fails, check:

82. What exactly does $dp[i]$ mean?
 83. Does it mean ending at i or best up to i ?
 84. Are base cases correct?
 85. Does transition use already-computed states?
 86. Is loop direction correct?
 87. Are impossible states initialized correctly?
 88. Is final answer $dp[n]$, $dp[n-1]$, or $\max(dp)$?
 89. Can space optimization overwrite a needed value?
-

44. What to Master Before Chapter 36

Before moving to 2D / Grid Dynamic Programming, make sure you can:

- Define $dp[i]$ clearly
- Solve Fibonacci-style DP
- Solve Climbing Stairs
- Solve Min Cost Climbing Stairs
- Solve House Robber
- Distinguish ending-at- i vs best-up-to- i
- Solve Maximum Subarray
- Solve Maximum Product Subarray
- Solve Decode Ways
- Solve Word Break

- Solve Coin Change
 - Solve LIS $O(n^2)$
 - Understand amount-based DP
 - Understand prefix DP
 - Understand state-machine DP
 - Space-optimize simple 1D DP
-

45. Final Cheat Sheet

Fibonacci-Style

```
dp[i] = dp[i - 1] + dp[i - 2];
```

Choose / Skip

```
dp[i] = max(dp[i - 1], dp[i - 2] + nums[i]);
```

Ending Here

```
current = max(nums[i], current + nums[i]);  
best = max(best, current);
```

Amount Minimum

```
dp[0] = 0;  
  
for (int a = 1; a <= amount; a++) {  
    for (int coin : coins) {  
        dp[a] = min(dp[a], dp[a - coin] + 1);  
    }  
}
```

Prefix Possible

```
dp[0] = true;  
  
for (int i = 1; i <= n; i++) {  
    for (int j = 0; j < i; j++) {  
        if (dp[j] && valid(j, i)) {  
            dp[i] = true;  
        }  
    }  
}
```

LIS

```
if (nums[j] < nums[i]) {  
    dp[i] = max(dp[i], dp[j] + 1);  
}
```

46. Core Takeaway

1D DP solves linear problems where each state depends on earlier states.

The key is to define:

`dp[i]`

precisely.

Ask:

Does `dp[i]` mean best up to `i`?

Does `dp[i]` mean best ending at `i`?

Does `dp[i]` mean ways to reach `i`?

Does `dp[i]` mean minimum cost for amount `i`?

Once the state is clear, the transition usually follows.

1D DP is the foundation for harder DP patterns.

Many advanced DP problems are just 1D DP with:

- more choices
 - extra states
 - extra dimensions
 - optimized transitions
 - stricter constraints
-